

01 QUERY STRUCTURE

Skeleton
`SELECT [DISTINCT] expressions
FROM R [JOIN S ON condition]
[WHERE row_condition]
[GROUP BY keys]
[HAVING group_condition]
[ORDER BY expression [ASC|DESC]]
[LIMIT n];`

Logical processing order

`FROM` / `JOIN` -> `WHERE` -> `GROUP BY`
-> `HAVING` -> `SELECT`
-> `ORDER BY` -> `LIMIT`

! Window `ORDER BY` controls the window calculation, not the final output order. Add an outer `ORDER BY` when display order matters.

02 MYSQL SYNTAX

String operations

`LIKE` 'abc%' -- %: any length; _: one character
`CONCAT`(a,b) -- || is logical OR by default in MySQL
`UPPER`(s); `LOWER`(s);
`CHAR_LENGTH`(s); `LENGTH`(s); -- characters vs bytes
`SUBSTRING`(s,pos,len);

COND `LIKE` case sensitivity depends on collation; MySQL is commonly case-insensitive.

! String positions start at 1: `SUBSTRING`(s,0,1) returns empty; use `SUBSTRING`(s,1,1) for the first character.

Common MySQL data types

`TINYINT`; `SMALLINT`; `MEDIUMINT`;
`INT`/`INTEGER`; `BIGINT`; `[UNSIGNED]`
`DECIMAL`(p,s); `NUMERIC`(p,s);
`FLOAT`; `DOUBLE`;
`CHAR`(n); `VARCHAR`(n); `TEXT`;
`BINARY`(n); `VARBINARY`(n); `BLOB`;
`DATE`; `TIME`; `DATETIME`; `TIMESTAMP`; `YEAR`;
`BOOLEAN`/`BOOL` = `TINYINT`(1); `JSON`; `ENUM`(...);
`SET`(...);

COND `DECIMAL`/`NUMERIC` are exact; `FLOAT`/`DOUBLE` approximate. `UNSIGNED` excludes negatives.

COND `CHAR` is fixed-width; `VARCHAR` is variable-width. `TEXT` stores long text; `BLOB` stores bytes.

Date/time functions

`CURDATE`()/`CURRENT_DATE`(); `CURTIME`()/`CURRENT_TIME`();
`NOW`()/`CURRENT_TIMESTAMP`(); `UTC_TIMESTAMP`();
`YEAR`(d); `QUARTER`(d); `MONTH`(d); `DAY`(d);
`DAYOFYEAR`(d);
`DAYNAME`(d); `MONTHNAME`(d); `LAST_DAY`(d);
`DAYOFWEEK`(d) Sun=1; `WEEKDAY`(d) Mon=0;
`HOUR`(t); `MINUTE`(t); `SECOND`(t); `EXTRACT`(unit
FROM dt);
`DATE`(dt); `TIME`(dt);
`DATE_ADD`(d, INTERVAL n DAY); `DATE_SUB`(d,
INTERVAL n MONTH);
`TIMESTAMPADD`(unit,n,dt); `DATEDIFF`(end,start);
`TIMESTAMPDIFF`(unit,start,end);
`ADDTIME`(t,'00:30:00'); `SUBTIME`(t,'00:30:00');
`TIMEDIFF`(end,start); `TIME_TO_SEC`(t);
`SEC_TO_TIME`(n);
`DATE_FORMAT`(dt,'%Y-%m-%d'); `STR_TO_DATE`(s,'%d
/%m/%Y');

COND `DATEDIFF`(end,start) returns days and ignores time; `TIMESTAMPDIFF` returns the chosen unit. `INTERVAL` supports date/time arithmetic.

! MySQL `TIMESTAMP` converts by session time zone; `DATETIME` preserves the stored date/time value.

03 NULL / AGGREGATION

x = NULL is wrong; use x IS NULL. Comparisons with NULL yield UNKNOWN; WHERE keeps only TRUE. Arithmetic with NULL usually returns NULL.

Counts `COUNT`(*) counts output rows; `COUNT`(x) ignores NULL; `COUNT`(`DISTINCT` x) counts unique non-NULL values.

COND Every selected nonaggregate expression must be grouped in standard SQL. WHERE filters rows before grouping; HAVING filters groups after aggregation.

! After a `LEFT JOIN`, `COUNT`(*) still counts a NULL-extended row; `COUNT`(B.id) ignores it.

04 JOIN / SET / SUBQUERY

Example data A(id,name): (1,Ana),(2,Bo);
B(id,tag): (1,DB),(3,Al). Match on id; output is (name,tag).

04 JOIN / SET / SUBQUERY (CONT.)

JOIN	Output (name,tag)
INNER JOIN	(Ana,DB)
CROSS JOIN	(Ana,DB), (Ana,Al), (Bo,DB), (Bo,Al)
LEFT JOIN	(Ana,DB), (Bo,NULL)
RIGHT JOIN	(Ana,DB), (NULL,Al)
FULL OUTER	(Ana,DB), (Bo,NULL), (NULL,Al)

COND Bare `JOIN` defaults to `INNER JOIN`; MySQL has no direct `FULL OUTER JOIN` syntax.

ON vs WHERE

A `LEFT JOIN B ON A.id=B.id AND B.tag='DB'`
The `ON` filter keeps Bo with NULL; putting B.tag='DB' in `WHERE` removes Bo.

Join traps `NATURAL JOIN` matches every same-named column, which can join unrelated fields; `USING`(k) names shared keys explicitly. Anti-join: `LEFT JOIN`, then `WHERE` the right key IS NULL.

Set operations `UNION` / `INTERSECT` / `EXCEPT` require union-compatible columns. They remove duplicates by default; `UNION ALL` preserves duplicates.

Subqueries

`WHERE x IN (SELECT ...)`
`WHERE EXISTS (SELECT 1 FROM S WHERE ...)`
`WHERE NOT EXISTS (...)`

Correlated subqueries refer to outer-row aliases. For “for all”, test that there is no missing requirement (e.g. required set EXCEPT possessed set is empty).

! NOT IN with any NULL in its subquery can yield UNKNOWN; NOT EXISTS is safer for anti-joins.

05 DDL / DML

CREATE / UPDATE / DELETE

`CREATE TABLE R` (
id INT PRIMARY KEY,
x VARCHAR(20) NOT NULL,
dept VARCHAR(20),
CHECK (...),
FOREIGN KEY (dept) REFERENCES D(dept)
);
`INSERT INTO R(...) VALUES (...);`
`UPDATE R SET x = expr [, y = expr]`
`WHERE` condition;
`DELETE FROM R WHERE` condition;

! `UPDATE/DELETE` without `WHERE` affects every row. Preview target rows with `SELECT ... WHERE ... first`.

COND Foreign-key actions include `ON DELETE/UPDATE CASCADE`, `SET NULL`, and `RESTRICT/NO ACTION`.

CTE

`WITH X AS (SELECT ...)`
`SELECT ... FROM X;`

06 RANKING / WINDOWS

For scores 100, 100, 90:

Function	100	100	90
ROW_NUMBER	1	2	3
RANK	1	1	3
DENSE_RANK	1	1	2

`ROW_NUMBER` is sequential and unique; ties need a deterministic tie-breaker. `RANK` leaves gaps after ties; `DENSE_RANK` does not.

! NULL scores still get a rank and tie with each other. Their position varies by DBMS and sort direction; control NULL placement or filter them first.

`RANK`() `OVER` (
PARTITION BY dept
ORDER BY score DESC)

`PARTITION BY` restarts within each group; `ORDER BY` sets window order. Ranking on grouped results happens after grouping/aggregation.

Window form

`window_fn(expr) OVER` (
[PARTITION BY p] [ORDER BY o]
[ROWS frame])

Windows calculate per row without collapsing rows. Aggregate windows include SUM/AVG/COUNT/MIN/MAX; ranking functions are nonaggregate.

Frames

-- Running: first row through current
`SUM`(x) `OVER` (`ORDER BY` d `ROWS BETWEEN`
UNBOUNDED PRECEDING AND CURRENT ROW)

06 RANKING / WINDOWS (CONT.)

-- Moving: previous 2 plus current
`AVG`(x) `OVER` (`ORDER BY` d `ROWS BETWEEN`
2 PRECEDING AND CURRENT ROW)
-- Symmetric: previous, current, next
`AVG`(x) `OVER` (`ORDER BY` d `ROWS BETWEEN`
1 PRECEDING AND 1 FOLLOWING)

! UNBOUNDED PRECEDING/FOLLOWING anchors the frame at the partition start/end; finite n PRECEDING/FOLLOWING bounds move with the current row.

! `ROWS` counts physical rows. `RANGE` groups `ORDER BY` peers, so ties can enlarge the frame.

07 RECURSIVE CTE

Use for transitive closure, hierarchies, and paths.

`WITH RECURSIVE` reach(a,b) `AS` (
-- Anchor: base edges
SELECT a,b `FROM` edge
UNION
-- Recursive term extends prior rows
SELECT r.a,e.b `FROM` reach r
JOIN edge e `ON` r.b=e.a
)
`SELECT * FROM` reach;

The anchor seeds rows; the recursive term refers to the CTE and repeatedly extends it.

COND `UNION` removes duplicates and can help closure/cycle control. `UNION ALL` avoids deduplication and needs a safe termination strategy.

08 PROCEDURES / TRIGGERS

MySQL stored procedure

`DELIMITER //`
`CREATE PROCEDURE p`(IN p_id INT, OUT p_cnt INT)
`BEGIN`
DECLARE v INT DEFAULT 0;
SELECT `COUNT`(*) INTO v `FROM` T `WHERE` id=p_id;
SET p_cnt=v;
`END//`
`DELIMITER ;`
`CALL` p(7,@n); `SELECT` @n;

IN receives a caller value; OUT returns one. Declare locals/conditions/handlers before executable statements. `SELECT ... INTO` stores a scalar result. `DELIMITER` is a client command so internal semicolons stay in the procedure body.

Triggers Fire on `INSERT`, `UPDATE`, or `DELETE`. `BEFORE` validates or changes `NEW`; `AFTER` runs after the row operation. This `BEFORE UPDATE` example validates a score, derives its label, and audits changes.

`DROP TRIGGER IF EXISTS`
trigger_before_update_safety_score;
`DELIMITER //`
`CREATE TRIGGER` trigger_before_update_safety_score
`BEFORE UPDATE ON` chicago_school
`FOR EACH ROW`
`BEGIN`
IF `NEW`.safety_score IS NOT NULL
AND (`NEW`.safety_score < 1 OR `NEW`.
safety_score > 99) THEN
SIGNAL `SQLSTATE` '45000'
SET `MESSAGE_TEXT` = 'safety_score must
be between 1 and 99 or NULL';
`END IF`;

IF `NEW`.safety_score IS NULL THEN
SET `NEW`.safety_icon = NULL;
ELSEIF `NEW`.safety_score < 20 THEN
SET `NEW`.safety_icon = 'Very Weak';
ELSEIF `NEW`.safety_score < 40 THEN
SET `NEW`.safety_icon = 'Weak';
ELSEIF `NEW`.safety_score < 60 THEN
SET `NEW`.safety_icon = 'Average';
ELSEIF `NEW`.safety_score < 80 THEN
SET `NEW`.safety_icon = 'Strong';
ELSE
SET `NEW`.safety_icon = 'Very Strong';
`END IF`;

IF NOT (`OLD`.safety_score <=> `NEW`.safety_score
) THEN
INSERT INTO school_safety_audit (
school_id,
old_safety_score,
new_safety_score
)
VALUES (
OLD.school_id,
OLD.safety_score,
NEW.safety_score
);
`END IF`;
`END //`
`DELIMITER ;`

Row images: `INSERT` uses `NEW`; `UPDATE` uses `OLD` and `NEW`; `DELETE` uses `OLD`. `SIGNAL` raises a user error; a trigger error aborts the invoking statement.

! Audit only changed values: test NOT (`OLD.v` <=> `NEW.v`) in an `UPDATE` trigger. NULL-safe equality avoids logging two NULLs as a change; `rollback` removes the audit row too.

09 PIVOT / CUBE / ROLLUP

Conditional aggregation Rows become grouping keys; `CASE` values become output columns.

General PIVOT pattern

`SELECT` row_dimension,
AGG(`CASE WHEN` pivot_column = 'val1'
THEN measure ELSE 0 `END`) `AS` val1,

AGG(`CASE WHEN` pivot_column = 'val2'
THEN measure ELSE 0 `END`) `AS` val2
`FROM` table_name
`GROUP BY` row_dimension;
! `PIVOT` = `GROUP BY` rows + `CASE WHEN` columns
+ AGG cells.
Example
`SELECT` location,
SUM(`CASE WHEN` type='THEFT' THEN 1 ELSE 0 `END`)
theft,
SUM(`CASE WHEN` type='BATTERY' THEN 1 ELSE 0
END) battery
`FROM` crime `GROUP BY` location;

For amounts, replace 1 with amount. `COUNT`(`CASE WHEN` condition THEN 1 `END`) also counts matches because `COUNT` ignores NULL.

Grouping sets For dimensions (a,b,c):

- CUBE(a,b,c): every subset, 2³ = 8 sets.
- ROLLUP(a,b,c): prefix hierarchy (abc, ab, a, empty), detail to subtotals to total.
- GROUPING(a)=1 when a is omitted in a subtotal/total; use it to distinguish generated NULL from data NULL.

MySQL supports `ROLLUP`. `CUBE`/general grouping-set syntax is commonly shown with PostgreSQL in course material.

10 LAST-MINUTE TRAPS

! Window functions preserve rows; `GROUP BY` collapses them. `RANK` leaves gaps; `DENSE_RANK` does not. `ROWS` counts physical rows; `RANGE` can include peers.

! Recursive CTE needs an anchor and termination. Trigger images: `INSERT`=`NEW`, `DELETE`=`OLD`, `UPDATE`=both. `GROUPING`() distinguishes subtotal NULL.

Query check Verify types/NULL/keys; row grain, join keys, and `ON` vs `WHERE`; grouping and `WHERE/HAVING`; window partition/order/frame and outer filter; final sort, rounding, aliases, column order, and `LIMIT`.

11 DBMS / RELATIONAL MODEL

Why a DBMS? Controls redundancy/inconsistency, difficult access, isolation, buried integrity rules, partial updates, concurrency, and authorization.

Abstraction Physical: how data is stored. Logical: which data and relationships exist. View: a user-specific representation.

COND Physical data independence means changing physical storage without changing the logical schema or applications.

Schema / instance Schema is structure; instance is a snapshot of tuples. A relation is a set of tuples (order is irrelevant); a domain is a set of allowed atomic values. NULL denotes unknown/missing.

Keys Superkey uniquely identifies tuples; candidate key is a minimal superkey; primary key is the selected candidate key. A foreign key references a key in another relation; NULL is allowed only if the column permits it.

12 RELATIONAL ALGEBRA

Rows	$\sigma_{condition}(R)$
Columns	$\pi_{columns}(R)$
All pairs	$R \times S$
Matched pairs	$R \bowtie_{condition} S$
Either set	$R \cup S$
Both sets	$R \cap S$
R but not S	$R - S$
Rename	$\rho_X(R)$
Assign result	$X \leftarrow E$

RA to SQL σ = WHERE; π = SELECT DISTINCT; $\bowtie$ = JOIN ... ON; $\cup$ = UNION; $-$ = EXCEPT/anti-join; ρ = alias AS.

! RA projection removes duplicates. SQL SELECT preserves them unless DISTINCT is requested.

13 PHP / HTML SYNTAX

PHP patterns

```
$x // variable
$s = $a . $b; // concatenate strings
foreach ($xs as $x) { ... }
$row["column"] // associative-array field
$_GET["x"]; $_POST["x"];
$conn = mysqli_connect(...);
$r = mysqli_query($conn, $sql);
$rows = mysqli_fetch_all($r, MYSQLI_ASSOC);
if ($_SERVER["REQUEST_METHOD"] === "POST") {
    ...
}
```

HTML form / dropdown

```
<form method="post">
  <select name="x">
    <option value="...">...</option>
  </select>
</form>
```

14 HTTP / MYSQL RESULTS

Request flow Browser request → web server → PHP interpreter → MySQL. The result returns to PHP, which creates HTML/text for the browser.

Query result mysqli_query() returns a result handle/object, not a PHP row array. Fetch rows with mysqli_fetch_all(\$r, MYSQLI_ASSOC).

Associative arrays key => value; with MYSQLI_ASSOC, database column names become keys, e.g. \$row["ID"].

GET / POST \$_GET comes from URL query parameters; \$_POST comes from the request body. \$_SERVER["REQUEST_METHOD"] identifies the method received by the page.

COND Use POST for form-body submissions; reading a variable from \$_GET/\$_POST is separate from validating its value.

15 SCHEMA REFINEMENT

Functional dependency X → Y: rows with the same X must have the same Y. Compute closure X+: start at X; repeatedly add Y when an FD's left side is already in X+.

Key test X is a superkey iff X+ contains every attribute. It is a candidate key iff no proper subset is a superkey.

Why normalize? Repeated facts cause update anomalies; missing facts can block inserts; deletes can erase unrelated facts.

3NF / BCNF / 4NF For each nontrivial FD X → A: 3NF if X is a superkey OR A is prime (in some candidate key). BCNF requires every determinant X to be a superkey. 4NF requires every nontrivial MVD X →» Y to have a superkey determinant.

15 SCHEMA REFINEMENT (CONT.)

! BCNF is stricter than 3NF; decomposing to BCNF can lose dependency preservation.

16 DECOMPOSITION / ER

Lossless join Decompose R into R1, R2 without spurious rows if (R1 intersect R2) → R1 OR (R1 intersect R2) → R2 follows from the FDs.

Dependency preserving Projected FDs on the parts must jointly imply the original FDs, so constraints can be checked without joining. 3NF synthesis usually preserves dependencies; BCNF may not.

ER to relations 1:1: FK on either side with UNIQUE (and NOT NULL if mandatory). 1:N: FK on the N side. M:N: bridge table with both FKs (often composite PK) plus relationship attributes. Weak entity: PK = owner's PK + partial key.

17 VIEWS / TRANSACTIONS

View

```
CREATE VIEW v AS SELECT ...;
```

Simple single-table views without grouping, aggregates, or DISTINCT are usually updatable. WITH CHECK OPTION rejects updates that would make a row disappear from the view. A materialized view stores results and can become stale; MySQL emulates one with a table plus refresh logic.

Transaction

```
START TRANSACTION; ... COMMIT;
ROLLBACK;
```

COND MySQL autocommit is ON by default. Use a transactional engine such as InnoDB; rollback only undoes work not already committed.

! Trigger writes share the caller's transaction: rolling back the data change also removes its audit row.

18 SQL EDGE PATTERNS

Conditional / NULL tools

```
CASE WHEN c THEN x ELSE y END
COALESCE(a,b); NULLIF(a,b); CAST(score AS UNSIGNED)
```

COALESCE returns first non-NULL; NULLIF returns NULL when equal. Use CAST before numeric comparison of digit strings.

Subquery cardinality Scalar subquery: one column, at most one row. x > ANY (subquery) needs one match; x > ALL (subquery) needs every comparison true. = ANY is equivalent to IN. For an empty set, ANY is false and ALL is true.

Symmetric difference Rows present in exactly one set:

```
(A EXCEPT B) UNION (B EXCEPT A)
```

Both inputs need compatible columns.

19 ENGINE / INDEX / ACCESS

DBMS path Query processor parses, optimizes, and executes SQL; storage manager handles files, buffers, indexes, and transactions; catalog stores schema and statistics.

Index B+ tree typically supports equality, ranges, and ordered scans; hash typically supports equality only. Indexes speed reads/joins/order but use space and slow writes.

Privileges

```
GRANT SELECT ON db.t TO 'u'@'host';
REVOKE SELECT ON db.t FROM 'u'@'host';
```

20 STORED PROGRAMS

Function / procedure Function returns one value for an expression; procedure uses CALL and may take IN/OUT parameters. Declare variables, cursors, and handlers before executable SQL; DECLARE ... HANDLER FOR NOT FOUND handles cursor exhaustion. Use IF/CASE for branches and WHILE, REPEAT, or LOOP for repetition.

Example: nearby crime records This procedure uses IN inputs and SELECT ... INTO, then a CTE computes distance for radius filtering and ordered output.

```
DROP PROCEDURE IF EXISTS
find_nearby_crime_records;
DELIMITER //
CREATE PROCEDURE find_nearby_crime_records(
  IN input_school_id INT,
  IN input_r DOUBLE
)
BEGIN
  DECLARE school_lat_rad DOUBLE;
  DECLARE school_lon_rad DOUBLE;
  SELECT RADIANS(latitude),
         RADIANS(longitude)
  INTO school_lat_rad,
         school_lon_rad
  FROM chicago_school
  WHERE school_id = input_school_id;
  WITH crime_rad AS (
    SELECT id, primary_description,
           crime_time,
           latitude, longitude,
           RADIANS(latitude) AS crime_lat_rad,
           RADIANS(longitude) AS
             crime_lon_rad
    FROM chicago_crime
    WHERE latitude IS NOT NULL AND longitude
           IS NOT NULL
  ),
  haversine_a AS (
    SELECT *,
           POW(SIN((school_lat_rad -
             crime_lat_rad)/2), 2)
             + COS(school_lat_rad) * COS(
               crime_lat_rad)
             * POW(SIN((school_lon_rad -
             crime_lon_rad)/2), 2) AS a
    FROM crime_rad
  ),
  crime_distance AS (
    SELECT id, primary_description,
           crime_time,
           latitude, longitude,
           ROUND(2 * 6371 * ASIN(SQRT(a)), 8)
           AS distance
    FROM haversine_a
  )
  SELECT id, primary_description, crime_time,
         latitude, longitude, distance
  FROM crime_distance
  WHERE distance <= input_r
  ORDER BY crime_time DESC, distance ASC, id
  ASC;
END //
DELIMITER ;
```

💡 6371 means distance and input_r are in km; exclude rows with missing coordinates.